Day 1: 神经网络速览 | LLM 基础课程

Day 1: 神经网络速览
- 开篇
- 1. 神经网络是什么?
  - 1.1 一句话定义
  - 1.2 最简单的神经网络
  - 1.3 为什么需要非线性?
- 2. 为什么神经网络能 work?
  - 2.1 理论基础：Universal Approximation Theorem
  - 2.2 深度的力量：为什么要”深”?
  - 2.3 优化：梯度下降为什么能找到好的解?
- 3. 关键技术细节
  - 3.1 激活函数的演进
  - 3.2 初始化的重要性
  - 3.3 正则化：防止过拟合
- 4. 代码示例：从零实现一个神经网络
- 5. 数学推导 [选读]
  - 5.1 前向传播的矩阵形式
  - 5.2 反向传播的推导
  - 5.3 为什么 Xavier 初始化有效?
- 6. 常见误解
  - ❌ “神经网络模拟人脑”
  - ❌ “深度学习是黑箱，没人知道为什么 work”
  - ❌ “参数越多越好”
- 7. 延伸阅读
  - 入门级
  - 进阶级
  - 论文
- 思考题
- 小结

Day 1: 神经网络速览

> **核心问题**：为什么深度学习能 work?

开篇

2012 年 9 月 30 日，ImageNet 大规模视觉识别挑战赛（ILSVRC）公布结果。一个叫 AlexNet 的神经网络以 15.3% 的 top-5 错误率夺冠，第二名是 26.2%——差距大到评委以为是作弊。

这不是神经网络第一次出现。早在 1980 年代，反向传播算法就已经发明了。但算力不够、数据不够、技巧不够，神经网络在 1990 年代陷入"AI 寒冬"，被 SVM 等方法压制了二十年。

2012 年之后，一切都变了。GPU 算力爆发、ImageNet 提供了百万级标注数据、ReLU 激活函数解决了梯度消失——深度学习复兴，然后是 AlphaGo，然后是 BERT，然后是 GPT-3，然后是 ChatGPT。

十几年后的今天，ChatGPT 能写代码、能聊天、能翻译，背后依然是神经网络——只是更大（千亿参数）、更深（上百层）、架构更精巧（Transformer）。

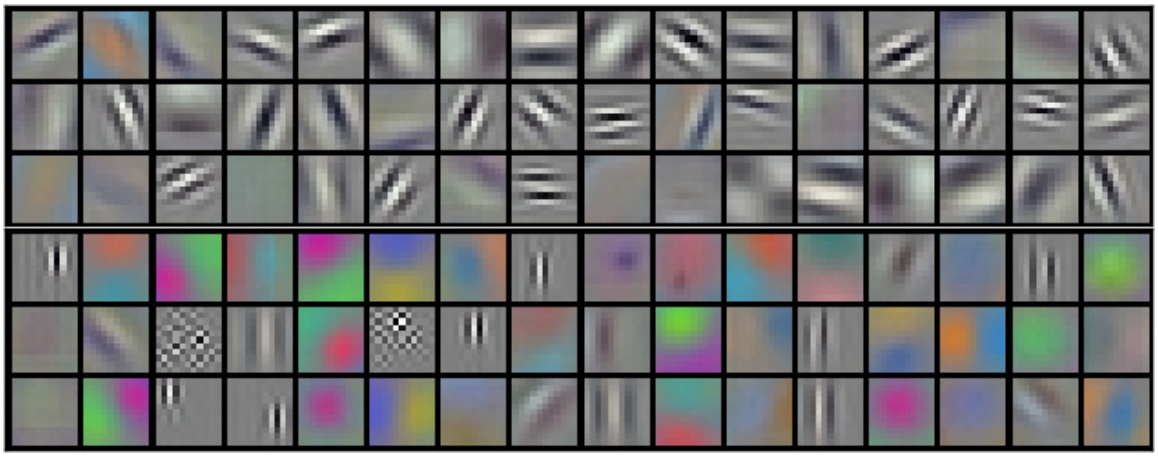

图 1：AlexNet 第一层卷积学习到的 96 个滤波器。上半部分是边缘检测器（不同方向的线条），下半部分是颜色和纹理检测器。这些是网络自动从数据中学到的，不是人工设计的。（来源：Krizhevsky et al., 2012）

但问题是：**为什么一堆矩阵乘法和非线性函数堆在一起，就能"理解"语言、"看懂"图片?**

这不是玄学。今天我们从数学和直觉两个角度，建立对神经网络的基础认知。

---

1. 神经网络是什么？

1.1 一句话定义

神经网络是一个可学习的函数逼近器（learnable function approximator）。

给它输入 x，它输出 y = f(x; θ)。其中 θ 是参数（可能有几十亿个）。训练的过程就是调整 θ，让 f 在你关心的任务上表现得越来越好。

这个定义很重要。它告诉我们： – 神经网络本质上是一个**函数** – 这个函数的形状由**参数**决定 – 训练 = **搜索**最优参数

1.2 最简单的神经网络

最简单的神经网络是**多层感知机**（MLP, Multi-Layer Perceptron），长这样：

```
输入层（d 维）  →  隐藏层（h 维）  →  输出层（k 维）
     x          →    h = σ(Wx + b)    →    y = softmax(Vh + c)
```

让我们拆解每一步：

第一步：线性变换

```
z = Wx + b
```

- W 是权重矩阵，形状 (h, d)
- b 是偏置向量，形状 (h,)
- 这一步把 d 维输入映射到 h 维

第二步：非线性激活

```
h = σ(z)
```

- σ 是激活函数（比如 ReLU: σ(x) = max(0, x)）
- 这一步引入非线性——**没有这一步，多层网络等价于单层**

第三步：输出层

```
y = softmax(Vh + c)
```

- V 是另一个权重矩阵，形状 (k, h)
- softmax 把输出变成概率分布（所有元素加起来等于 1）

就这么简单。复杂的网络无非是： 1. 把这个结构叠很多层（深度） 2. 每层有很多神经元（宽度） 3. 在连接方式上做文章（卷积、注意力等）

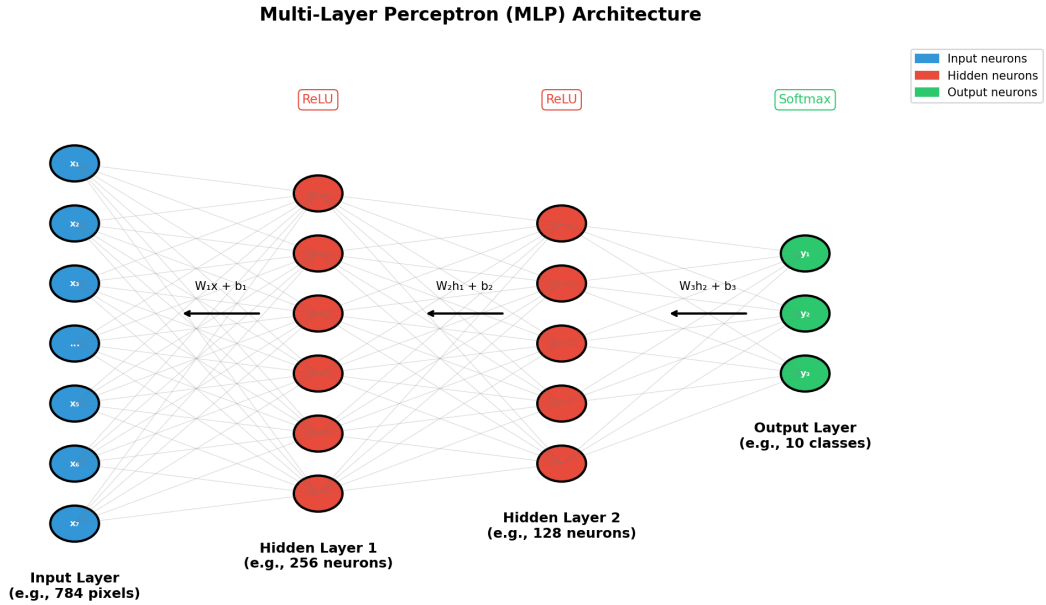

图 2：多层感知机（MLP）架构。输入层接收原始数据，隐藏层通过权重矩阵和激活函数提取特征，输出层给出最终预测。

1.3 为什么需要非线性?

这是一个关键问题。假设我们有两层线性变换：

```
h = W₁x
y = W₂h = W₂W₁x = Wx
```

两层线性变换等价于一层！因为矩阵乘法是线性的，W_2W_1 可以合并成一个矩阵 W。

这意味着：**无论你叠多少层线性变换，最终都只能表示线性函数。**

但现实世界的问题几乎都是非线性的。比如图像分类："这张图是猫还是狗"不是像素值的线性组合能判断的。

所以我们需要在每层之间插入非线性激活函数。这样，多层网络就能表示复杂的非线性函数。

---

2. 为什么神经网络能 work?

这是最核心的问题。我们从三个角度回答。

2.1 理论基础：Universal Approximation Theorem

1989 年，George Cybenko 证明了一个里程碑式的定理：

> Universal Approximation Theorem：一个单隐藏层的神经网络，只要隐藏层神经元足够多，就能以任意精度逼近定义在紧集上的任何连续函数。

用人话说：只要神经元够多，神经网络可以拟合任意复杂的函数。

直觉理解：

想象你要逼近一个复杂的曲线。每个神经元可以看作一个"阶梯函数"或"小山包"。用足够多的小山包叠加，你可以逼近任何形状。

这就像傅里叶变换：任何函数都可以分解成正弦波的叠加。神经网络用的不是正弦波，而是激活函数，但思想类似。

但要注意：

这个定理只说"存在这样的网络"，没说： 1. 你需要多少神经元（可能是天文数字） 2. 你能不能训练出来（优化可能很难） 3. 它能不能泛化到新数据（可能过拟合）

理论上的可能性和实践中的可行性是两回事。

2.2 深度的力量：为什么要"深"？

既然单层就能逼近任意函数，为什么还要深度？

答案是**效率**。

例子 1：异或问题

计算 n 个变量的异或（XOR）： – 浅层网络需要 $O(2^n)$ 个神经元 – 深层网络只需要 $O(n)$ 个神经元

差距是指数级的。

例子 2：图像特征层次

识别一张人脸图像： – 第 1 层：检测边缘（水平线、垂直线、斜线） – 第 2 层：组合边缘成简单形状（角、弧） – 第 3 层：组合形状成部件（眼睛、鼻子、嘴巴） – 第 4 层：组合部件成人脸

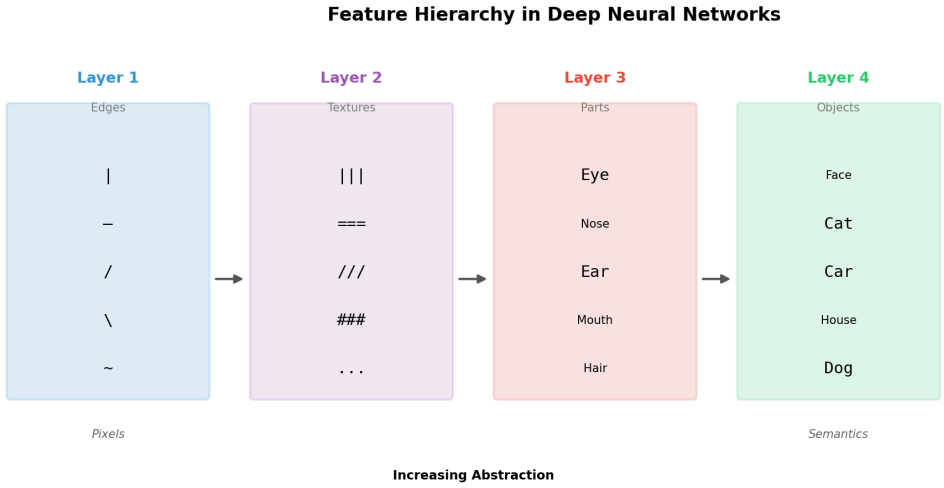

图 3：深度网络的特征层次。第一层检测边缘，第二层组合成纹理，第三层识别部件，第四层识别完整对象。每一层都在前一层的基础上构建更抽象的表示。

每一层都在前一层的基础上构建更抽象的特征。如果用单层网络，你需要直接从像素跳到"人脸"，这需要海量的神经元来编码所有可能的组合。

数学上的解释：

深度网络具有**组合爆炸**的表达能力。假设每层有 n 个神经元，L 层深度： – 浅层网络（1 层）：可以表示 O(n) 种模式 – 深层网络（L 层）：可以表示 $O(n^L)$ 种模式

深度带来的是**指数级**的表达能力增长。

2.3 优化：梯度下降为什么能找到好的解？

有了强大的函数逼近能力，还需要能训练出来。这靠的是**梯度下降**（Gradient Descent）+ **反向传播**（Backpropagation）。

梯度下降的核心思想：

想象你站在山上，想走到最低点（损失最小的地方），但你蒙着眼睛。怎么办？

一个直觉的方法：用脚试探周围哪个方向是下坡的，然后往那个方向走一小步。重复这个过程，最终你会走到某个低点。

数学上：

```
θ_new = θ_old − η · ∇L(θ)
```

- θ 是参数
- L(θ) 是损失函数（越小越好）
- ∇L(θ) 是梯度（指向损失增加最快的方向）
- η 是学习率（每一步走多大）

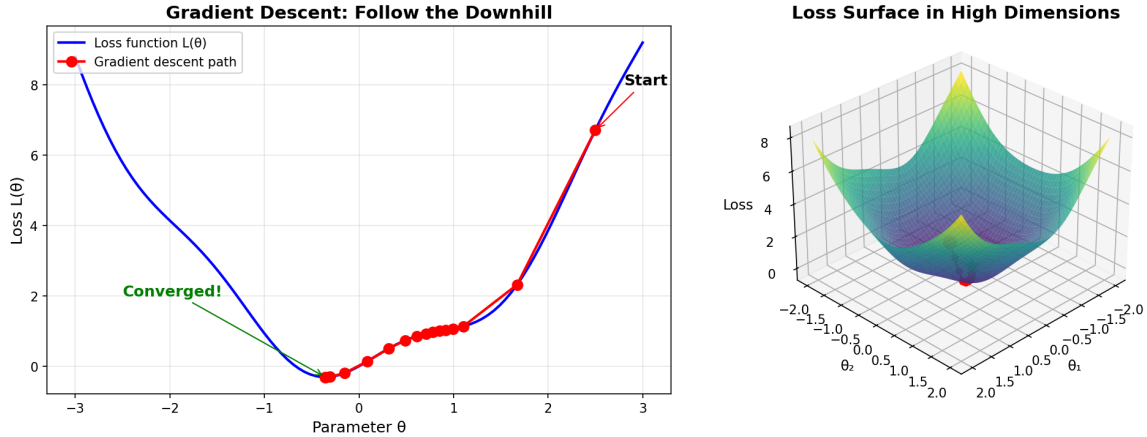

图 5：梯度下降可视化。左图展示 2D 情况下沿"下坡"方向迭代；右图展示高维损失曲面，红点是起始位置，绿星是收敛位置。

反向传播的作用：

神经网络可能有几十亿个参数。如果朴素地计算每个参数的梯度，复杂度是 O(参数数量^2)——根本不可行。

反向传播利用链式法则，把梯度计算复杂度降到 O(参数数量)。这是深度学习能 scale 的关键。

一个神奇的现象：

神经网络的损失函数是**非凸的**，有无数个局部最小值。理论上，梯度下降可能卡在任何一个局部最小值。

但经验上，在高维空间中，大多数局部最小值的损失都差不多好。而且，鞍点（saddle points）比局部最小值更常见，而 SGD 能逃离鞍点。

为什么会这样？这至今是一个开放的研究问题。但它 work，而且 work 得很好。

---

3. 关键技术细节

3.1 激活函数的演进

Sigmoid（1980s–2000s）

```
σ(x) = 1 / (1 + e^(-x))
```

输出在 (0, 1) 之间，有概率解释。

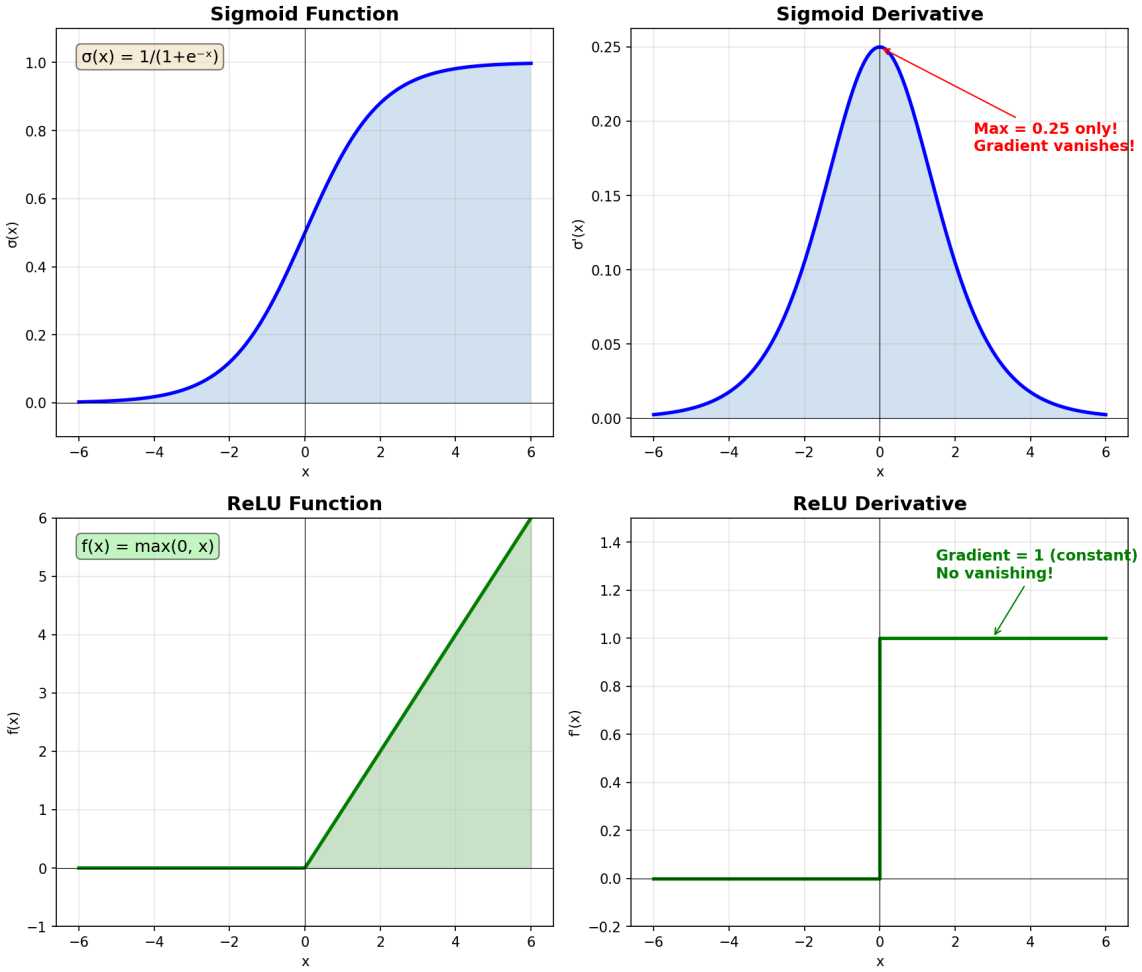

图 4：Sigmoid vs ReLU 激活函数对比。左列是函数本身，右列是导数。注意 Sigmoid 导数最大值只有 0.25，而 ReLU 在 x > 0 时导数恒为 1。

问题：当 |x| 很大时，σ'(x) ≈ 0。梯度在深层网络中会指数级衰减——这叫**梯度消失**（vanishing gradient），是深度网络训练的噩梦。

ReLU（2010s 至今）

```
ReLU(x) = max(0, x)
```

优点： – x > 0 时，梯度恒为 1，不会消失 – 计算简单，比 sigmoid 快 6 倍 – 稀疏激活：大约 50% 的神经元输出为 0，提供正则化效果

缺点： – x < 0 时，梯度为 0，神经元可能"死掉"

ReLU 变体 – LeakyReLU: max(0.01x, x)，让负半轴有小梯度 – GELU: x · Φ(x)，GPT 和 BERT 使用，更平滑 – SiLU/Swish: x · σ(x)，近年流行

3.2 初始化的重要性

参数的初始值对训练影响巨大。

错误的初始化： – 全部初始化为 0：所有神经元完全一样，无法学习 – 初始化太大：梯度爆炸 – 初始化太小：梯度消失

正确的初始化（Xavier/He）：

```
# Xavier 初始化（用于 sigmoid/tanh）
W = np.random.randn(fan_in, fan_out) * np.sqrt(1 / fan_in)

# He 初始化（用于 ReLU）
W = np.random.randn(fan_in, fan_out) * np.sqrt(2 / fan_in)
```

核心思想：让每层的输出方差保持稳定，不要越来越大或越来越小。

3.3 正则化：防止过拟合

神经网络参数很多，容易过拟合（在训练集上表现很好，在测试集上很差）。

Dropout 训练时随机"关掉"一部分神经元（比如 50%）。这迫使网络不能依赖任何单个神经元，提高鲁棒性。

```
# 训练时
h = h * (torch.rand_like(h) > 0.5)  # 随机置零
h = h * 2  # 缩放，保持期望不变

# 推理时
# 不做 dropout
```

Weight Decay (L2 正则化) 在损失函数中加入参数的 L2 范数：

```
L_total = L_task + λ · ||θ||²
```

这会惩罚太大的参数，让网络更"简单"。

---

4. 代码示例：从零实现一个神经网络

用 PyTorch 写一个完整的训练流程：

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# 1. 定义网络
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
```

```
            nn.Flatten(),                    # 28x28 -> 784
            nn.Linear(784, 512),             # 784 -> 512
            nn.ReLU(),
            nn.Dropout(0.2),                 # 防止过拟合
            nn.Linear(512, 256),             # 512 -> 256
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 10)               # 256 -> 10 (数字 0-9)
        )

    def forward(self, x):
        return self.layers(x)

# 2. 准备数据 (MNIST 手写数字)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))  # 标准化
])

train_dataset = datasets.MNIST('./data', train=True, download=True,
        transform=transform)
test_dataset = datasets.MNIST('./data', train=False,
        transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000)

# 3. 训练
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = MLP().to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

for epoch in range(5):
    model.train()
    total_loss = 0
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad()        # 清空梯度
        output = model(data)         # 前向传播
        loss = criterion(output, target)  # 计算损失
        loss.backward()              # 反向传播
        optimizer.step()             # 更新参数

        total_loss += loss.item()

    # 测试
    model.eval()
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
```

```
            data, target = data.to(device), target.to(device)
            output = model(data)
            pred = output.argmax(dim=1)
            correct += pred.eq(target).sum().item()

    accuracy = 100. * correct / len(test_dataset)
    print(f'Epoch {epoch+1}: Loss={total_loss/len(train_loader):.4f},
        Test Accuracy={accuracy:.2f}%')

# 输出示例:
# Epoch 1: Loss=0.3521, Test Accuracy=96.12%
# Epoch 2: Loss=0.1423, Test Accuracy=97.45%
# Epoch 3: Loss=0.1024, Test Accuracy=97.89%
# Epoch 4: Loss=0.0812, Test Accuracy=98.01%
# Epoch 5: Loss=0.0673, Test Accuracy=98.15%
```

几点说明： 1. **nn.Sequential** 把多层串起来 2. **optimizer.zero_grad()** 很重要：PyTorch 默认累加梯度，每个 batch 前要清空 3. **model.train() / model.eval()** 切换模式：Dropout 只在训练时生效 4. **with torch.no_grad()** 推理时不需要计算梯度，省内存

---

5. 数学推导 [选读]

> 这部分面向想深入理解的读者，可跳过。

5.1 前向传播的矩阵形式

对于 L 层神经网络，令 $h^{(0)} = x$（输入），则：

$$\begin{aligned} \text{线性变换} \\ \text{激活} \\ \text{输出} \end{aligned}$$

5.2 反向传播的推导

设损失函数为交叉熵：

定义误差信号：

对于输出层（使用 softmax + 交叉熵的特殊性质）：

对于隐藏层（链式法则）：

参数的梯度：

5.3 为什么 Xavier 初始化有效?

设输入 x 有 n 个元素，每个元素方差为 Var(x)。权重 W 的每个元素独立同分布，均值为 0，方差为 Var(W)。

线性层输出：

输出方差：

为了让 Var(z) = Var(x)（方差不变），需要：

这就是 Xavier 初始化的来源。

---

6. 常见误解

❌ “神经网络模拟人脑”

这是个美丽的误会。现代神经网络和生物神经元的相似性仅限于”都是节点连接”这个抽象层面。

实际差异：| 方面 | 生物神经元 | 人工神经元 | |——|————–|————–| | 信号 | 脉冲（spike），时间相关 | 连续数值 | | 学习 | 突触可塑性，局部规则 | 反向传播，全局梯度 | | 结构 | 稀疏、不规则连接 | 通常全连接或规则结构 | | 能耗 | ~20W（整个大脑） | ~300W（一块 GPU） |

神经网络的成功不是因为它像大脑，而是因为： 1. 它是一个强大的函数逼近器 2. 我们有高效的训练算法（反向传播） 3. 我们有足够的数据和算力

❌ “深度学习是黑箱，没人知道为什么 work”

这说法半对半错。

我们**确实不完全理解**的： – 为什么非凸优化能找到好的解 – 为什么过参数化反而有助于泛化 – 神经网络内部学到了什么”表示”

我们**有扎实理论基础**的： – Universal Approximation Theorem 解释了表达能力 – 统计学习理论（VC 维、Rademacher 复杂度）解释了泛化 – Scaling Laws 给出了性能–规模的经验公式

“不完全理解”≠“完全不理解”。深度学习既有理论，也有大量未解之谜。

❌ “参数越多越好”

不一定。参数多了容易过拟合。

但有趣的是，当参数多到一定程度（“过参数化”），反而能泛化得更好。这叫 **double descent** 现象，是近年深度学习理论的热点。

经验法则：现代大模型确实越大越好（参见 Scaling Laws），但前提是有足够的数据和正则化。

---

7. 延伸阅读

入门级

1. **3Blue1Brown: Neural Networks** (视频)
   最直观的神经网络可视化解释
   https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

2. **Neural Networks and Deep Learning** (Michael Nielsen)
   交互式在线书籍，从零开始
   http://neuralnetworksanddeeplearning.com/

进阶级

3. **Deep Learning Book – Chapter 6** (Goodfellow et al.)
   深度学习圣经的神经网络章节
   https://www.deeplearningbook.org/contents/mlp.html

4. **CS231n: Convolutional Neural Networks** (Stanford)
   斯坦福经典课程，有详细的反向传播推导
   https://cs231n.stanford.edu/

论文

5. **Universal Approximation Theorem** (Cybenko, 1989)
   奠基性论文
   https://doi.org/10.1007/BF02551274

6. **ImageNet Classification with Deep CNNs** (Krizhevsky et al., 2012)
   AlexNet 论文，深度学习复兴的起点
   https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks

---

思考题

1. **ReLU 在 x < 0 时梯度为 0，这意味着某些神经元可能"死掉"（永远不更新）。这是 bug 还是 feature？有什么解决方案？**

2. **Universal Approximation Theorem 说单层网络就能逼近任意函数，但为什么实践中深层网络效果更好？除了"效率"之外，还有什么原因？**

3. **如果把神经网络的所有激活函数换成线性函数 σ(x) = x，网络会变成什么？还能学习复杂任务吗？**

小结

今天我们建立了神经网络的基础认知：

概念	一句话解释
神经网络	可学习的函数逼近器
Universal Approximation	理论上能逼近任意连续函数
深度的价值	用更少参数表示更复杂函数

概念	一句话解释
反向传播	高效计算梯度的算法
ReLU	解决梯度消失的激活函数
Dropout	防止过拟合的正则化技术
Xavier/He 初始化	保持各层方差稳定

关键 takeaway：神经网络不是魔法，是数学。它的成功来自强大的函数逼近能力 + 高效的优化算法 + 足够的数据和算力。

明天我们聊 RNN——神经网络处理序列数据的第一次尝试，以及它为什么最终被 Transformer 取代。

Day 1 of 60 | LLM Fundamentals 字数：约 4500 字 | 阅读时间：约 18 分钟